DEEP-LEARNING

UNIT-2

II	DEEP NETWORKS : History of Deep Learning- A Probabilistic Theory of Deep Learning- Backpropagation and regularization, batch normalization- VC Dimension and Neural Nets-Deep Vs Shallow Networks-Convolutional Networks- Generative Adversarial Networks (GAN), Semi-supervised Learning	

What is Deep Learning?

—> Deep learning is a type of artificial intelligence (AI) that tries to make computers learn like humans — by using "neural networks" (systems inspired by the human brain).

QUE:- Demonstrate deep learning. Explain its uses, application and history. (AKTU 22-23)

Early Days (1940s–1980s)

- **1940s–1950s**: The idea of deep learning started with scientists trying to copy how the human brain works. They created simple models called *artificial neurons* that could process information. A guy named Warren McCulloch and Walter Pitts made the first model of a neuron in 1943.
- **1950s–1960s**: A scientist named Frank Rosenblatt invented the *perceptron*, a simple system that could learn to recognize basic patterns, like shapes. This was exciting, but it could only solve very simple problems.
- **1980s**: The *backpropagation* algorithm was developed (by people like Geoffrey Hinton and others). This was a big deal because it let neural networks learn from their mistakes and get better at tasks. However, computers back then were slow, and deep learning didn't take off yet.

The Quiet Period (1990s–2000s)

- During the 1990s and early 2000s, deep learning wasn't very popular. Other AI methods, like simpler algorithms, worked better for the computers of that time. Neural networks needed a lot of data and computing power, which wasn't available yet. But researchers like Geoffrey Hinton, Yann LeCun, and Yoshua Bengio kept working on it, believing in its potential.

The Big Breakthrough (2000s–2010s)

- **2006–2010**: Things changed when computers got much faster, thanks to powerful chips called GPUs (graphics processing units). These chips could handle the math needed for deep learning. Also, the internet gave scientists tons of data (like images, videos, and text) to train their models.
- **2012**: A huge moment happened when a deep learning model called *AlexNet* (created by Alex Krizhevsky and team) won a big image recognition competition called ImageNet. It was way better than other methods at recognizing objects in pictures. This showed the world that deep learning was powerful.

Deep Learning Boom (2010s–Now)

- After AlexNet, deep learning became a superstar in AI. Companies like Google, Facebook, and Amazon started using it for things like:

Made with Napkin

- Translating languages (like Google Translate)
- Making self-driving cars
- Creating voice assistants like Siri and Alexa

- **2010s–2020s**: Researchers made neural networks deeper (more layers) and better, leading to models like GPT (for text) and DALL-E (for images). These models can write stories, create art, and even help doctors diagnose diseases.
- Today, deep learning is everywhere—your phone, apps, and even video games use it. It keeps getting better because of more data, faster computers, and new ideas from researchers.

A Probabilistic Theory of Deep Learning

A Probabilistic Theory of Deep Learning explain in very easy language in deep learning

Deep learning uses neural networks (like a computer's brain) to learn from data, such as recognizing cats in pictures or translating languages. The probabilistic theory says that deep learning is all about dealing with uncertainty. Instead of being 100% sure about something, the model makes smart guesses based on patterns it finds in data.

So instead of saying:

> "This image is a cat."

A deep learning model says:

> "There's a 95% chance this image is a cat, 4% dog, 1% rabbit."

This way, the model expresses **how sure** it is.

Key Points in Simple Terms

1. **Learning from Data**:
   - Deep learning models look at tons of examples (like thousands of cat pictures).
   - They figure out patterns, like "cats often have whiskers and pointy ears."
   - These patterns help the model guess what's in new pictures.

2. **Probability is Key**:
   - Instead of saying "This is definitely a cat," the model says, "There's an 80% chance this is a cat, 15% chance it's a dog, and 5% chance it's something else."
   - This "probability" approach helps the model handle messy or unclear data.

3. **Why Probability Helps**:
   - Real-world data is often tricky. For example, a blurry picture or a cat in a weird pose might confuse a model.
   - By using probabilities, the model can still make a good guess even if the data isn't perfect.

4. **How the Model Learns**:
   - The model starts with random guesses.
   - It uses a math trick called *backpropagation* to adjust its guesses by comparing them to the correct answers.
   - Over time, it gets better at predicting the right probabilities (like being more sure about cats vs. dogs).

5. **Deep Networks and Layers**:
   - Deep learning models have many layers (like stacking pancakes).
   - Each layer looks at different parts of the data (e.g., edges, shapes, or whole objects).

- The probabilistic theory explains how these layers work together to calculate probabilities for complex tasks, like recognizing a specific person's face.

Why Is This Theory Useful?

- It helps us understand *why* deep learning works so well for things like image recognition, speech translation, or even playing games.
- It gives researchers ideas to make models better, especially when there's not much data or when things are uncertain.

Regularization

Regularization is like telling the neural network, "Don't overcomplicate things!" It helps the network learn patterns that work well for new data, not just the data it's trained on.

Why It's Needed:

- Sometimes, a neural network learns *too well* on the training data (like memorizing answers for a test instead of understanding the subject).
- This is called *overfitting*—the network does great on the training data but fails on new, unseen data (like a new test).
- Regularization stops the network from becoming too complex or memorizing the data.

There are a few common ways to do this:

1. **L1/L2 Regularization**:
    - Adds a penalty for having big weights.
    - It's like saying, "Keep your answers simple, don't overthink it."
    - This makes the network focus on the most important patterns.
2. **Dropout**:
    - During training, randomly "turns off" some parts of the network (like ignoring some neurons).
    - It's like studying without relying on one specific friend for all the answers, so you learn to solve problems in different ways.
    - This helps the network work better on new data.
3. **Data Augmentation**:
    - Adds variety to the training data (like flipping pictures or adding noise).
    - It's like practicing with different versions of a test so you're ready for anything.

What is Batch Normalization?

Batch Normalization is an algorithm method which makes the training of deep neural network faster and more stable.

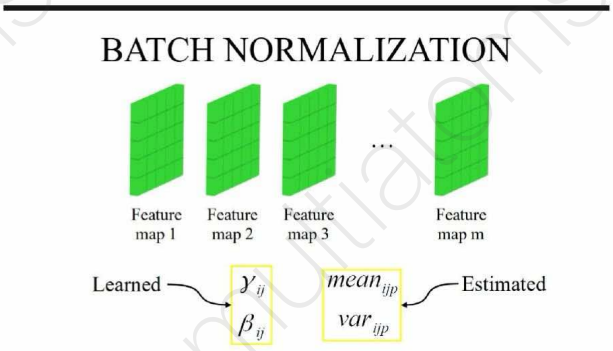
It consist of normalizing activation vectors from hidden layers using the mean and variance of the current batch. This normalization step is applied right before the nonlinear function .

What Batch Normalization Does

Batch Normalization:

1. Takes a small group of training examples (called a **mini-batch**).
2. Finds the **average (mean)** and **spread (variance)** of the numbers in that batch.
3. Adjusts the numbers so that they are centered around **0** and have a similar scale (like making all values balanced).
4. Then, it **scales and shifts** them slightly using two learnable values (γ and β), so the network can still adjust if needed.

This helps the network stay balanced and learn faster!



What is VC Dimension?

VC Dimension tells us **how powerful or flexible a model is** —basically, **how much a model can learn or fit different kinds of data.**

—> VC stands for Vapnik–Chervonenkis

It's named after two scientists, **Vladimir Vapnik** and **Alexey Chervonenkis**, who came up with this idea to measure the **capacity** (strength) of a machine learning model.

Why VC Dimension Matters

1. **Measures Model Capacity**It tells us how complex patterns a model can learn.

2. **Helps Avoid Overfitting**
   - If the VC dimension is **too small**, the model is too simple → **underfitting** (can't learn enough).
   - If the VC dimension is **too large**, the model might **memorize everything** → **overfitting** (learns too much).
3. **Tells How Much Data We Need**
   - A model with **high VC dimension** needs **more data** to learn properly.
   - A **smaller model** can learn well with less data.

Concept	Simple Meaning
VC Dimension	A number that measures how flexible a model is
High VC Dimension	Model can learn complex patterns but may overfit
Low VC Dimension	Model is simple and may underfit
Neural Networks	Have very high VC dimension because of many layers and parameters
Goal	Balance between flexibility (learning power) and generalization (avoiding overfitting)

QUE:- Draw and explain the architecture of Convolutional Networks. (AKTU 22-23)

Convolutional Networks

Made with Napkin

A **Convolutional Neural Network** (or **CNN**) is a special kind of neural network that is really good at **understanding images** , **videos**, or **anything with patterns** (like shapes, edges, or textures).

It's designed to look at pictures **the way humans do** — by focusing on small parts first, and then putting everything together to understand the whole image.

Think of it like this:

When you look at a photo of a cat 🐱:

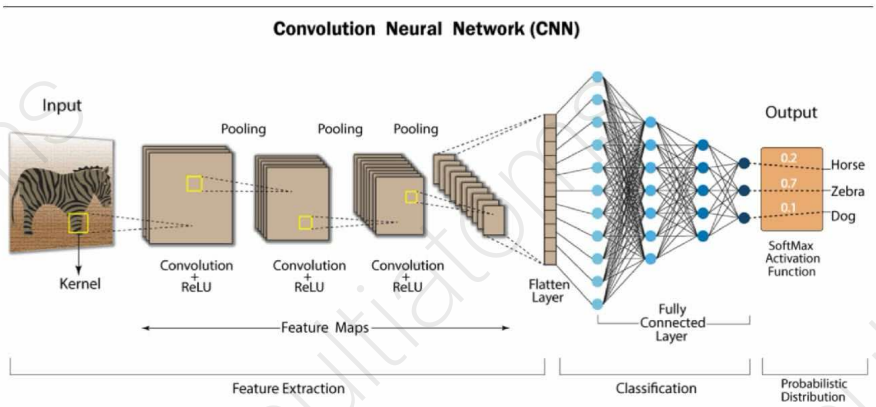
1. Your eyes first notice **edges and colors** (whiskers, ears, eyes).
2. Then your brain combines those details and says, "Aha, that's a cat!"

CNNs do the same thing:

- First, they look for **simple patterns** (like lines or corners).
- Then, they combine those into **bigger patterns** (like eyes, faces, or shapes).
- Finally, they decide **what the object is** (cat, dog, car, etc.).

Made with Napkin

multiatoms



Main Building Blocks of CNNs

1. Convolution Layer

This is the "heart" of a CNN.

- It uses **filters (small windows)** that move across the image.
- Each filter looks for specific features — for example:
  - One filter might detect edges,
  - Another might detect corners,
  - Another might detect bright spots.

2. ReLU Layer (Activation Function)

- After convolution, CNN applies something called **ReLU (Rectified Linear Unit)**.
- It simply keeps **positive values** and removes **negative ones** — making the image data easier to process and helping the network learn faster.

3. Pooling Layer (Downsampling)

- After finding features, we don't need every small detail.
- The **Pooling layer** reduces the size of the image but keeps the important information.

This makes the network smaller and faster, and it helps prevent **overfitting**.

4. Fully Connected Layer (Dense Layer)

- At the end, CNN flattens all the features it found (like eyes, nose, fur, etc.) into one long list.
- Then it connects to normal **neural network layers** that make the final decision — for example:
  - "This is 95% likely to be a cat"
  - "This is 4% likely to be a dog"

5. Output Layer

- This layer gives the **final prediction**, often using **softmax** (which converts outputs into probabilities that add up to 1).

QUE:- Explain GAN and its models. Name and describe different types of GANs. (AKTU 22-23)

Generative Adversarial Networks (GAN)

Made with Napkin

Generative Adversarial Networks (GANs) are a super cool type of deep learning model that can create new things, like pictures, music, or even text, that look real.

What Are GANs?

A GAN is like a team of two neural networks working together (and against each other):

- **Generator**: This one makes fake stuff, like creating a picture of a cat that doesn't exist.
- **Discriminator**: This one checks if the stuff is real (from actual data) or fake (made by the generator).

How Do GANs Work?

Here's the process in simple steps:

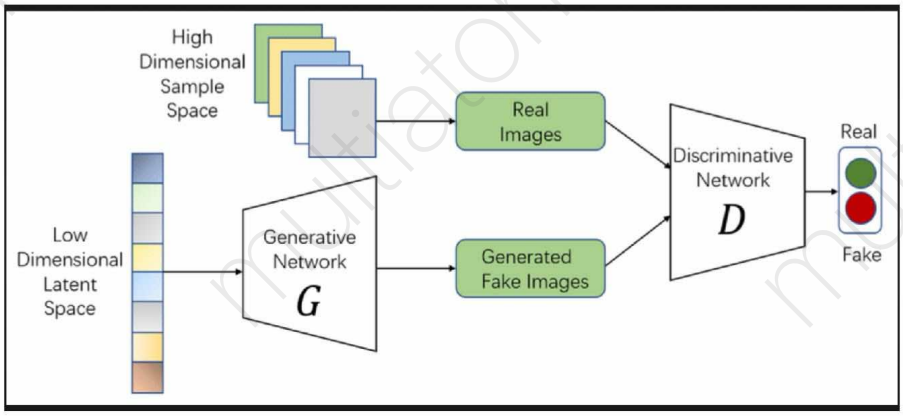
1. **Generator Creates Something Fake**:
    - The generator takes random noise (like a scribble) and turns it into something, like a picture of a cat.
    - At first, the fake cat looks messy or weird because the generator is still learning.

2. **Discriminator Judges It**:
    - The discriminator looks at the fake cat and real cat pictures from a dataset.
    - It decides, "Is this a real cat or a fake one?" (e.g., "This looks fake!").

3. **Learning from Feedback**:
    - If the discriminator spots the fake, it tells the generator, "Try harder!" The generator adjusts to make better fakes.
    - The discriminator also learns to get better at spotting fakes by comparing real and fake examples.
    - They use **backpropagation** (like fixing mistakes after a test) to improve.

4. **Keep Playing the Game**:
    - The generator keeps making fakes, and the discriminator keeps judging them.
    - Over time, the generator gets so good that the discriminator can't tell the difference between real and fake cats (e.g., it's 50/50 whether it's real).

5. **Result**:
    - The generator can now create super realistic images, like a cat that looks real but was made from scratch!



Where Are GANs Used?

- **Art and Photos**: Creating fake faces, turning sketches into realistic images, or making AI-generated artwork.
- **Video Games**: Designing characters or environments.
- **Movies**: Generating special effects or aging/de-aging actors.
- **Science**: Creating fake data for experiments, like simulating medical images.

Made with Napkin

- **Fashion**: Designing new clothes or hairstyles.

QUE:-1 Examine the Semi –Supervised learning. (AKTU 22-23)

Semi-supervised Learning

Semi-supervised learning mixes supervised learning (learning from examples with answers, like a teacher guiding you) and unsupervised learning (figuring things out without answers, like solving a puzzle). It uses a small amount of labeled data and a large amount of unlabeled data to teach a model to make good predictions.

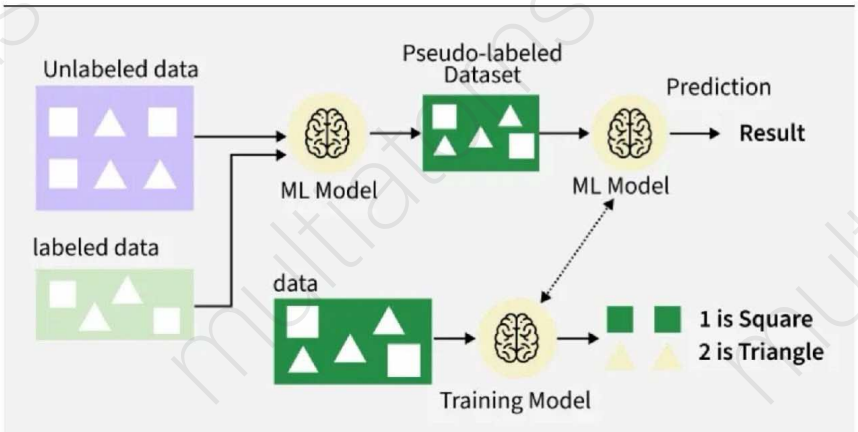
How Does It Work?

1. **Learn from Labeled Data**:
   - Start with a small set of data that has labels, like 10 photos tagged as "cat" or "dog."
   - The model learns patterns from these, like "cats have pointy ears."

2. **Guess Labels for Unlabeled Data**:
   - Look at a big pile of unlabeled data, like 100 untagged photos.
   - The model guesses labels for these based on what it learned (e.g., "This photo has pointy ears, so maybe it's a cat").

3. **Improve with Both**:
   - Use the labeled data and the guessed labels to keep training the model.
   - The model gets better by practicing with both, like a student doing homework after a few lessons.

4. **Repeat**:
   - Keep guessing and refining until the model is good at predicting labels for all data.



Why Use Semi-supervised Learning?

- **Saves Effort**: Labeling data (like tagging photos) is slow and costly. This method needs only a few labels.
- **Uses More Data**: It makes use of tons of unlabeled data, like social media posts or videos.
- **Better Results**: It often makes models smarter than using just a few labeled examples.

Where Is It Used?

Made with Napkin

- **Face Recognition**: Learning from a few tagged faces and lots of untagged photos.
- **Handwriting**: Recognizing different handwriting styles with a few labeled examples.
- **Speech**: Improving voice assistants with some labeled audio and lots of unlabeled recordings.

PYQs

QUE:-1 Explain GAN and its models. Name and describe different types of GANs.

ANS :-

1️⃣ Vanilla GAN (Basic GAN)

—> **What it is:**This is the **simplest and original version** of GAN — the one that started it all (created by Ian Goodfellow in 2014).

—> **How it works:**

- The **Generator** makes fake data (for example, fake numbers or simple images).
- The **Discriminator** tries to tell if it's real or fake.
- They train together and improve step by step.

—>**Example:**If you train it on thousands of handwritten digits (like the MNIST dataset), it can learn to **create new digits** that look like real handwriting — even though those digits were never written by anyone.

2️⃣ DCGAN (Deep Convolutional GAN)

—> **What it is:**"Deep Convolutional GAN" — it uses **Convolutional Neural Networks (CNNs)** inside the generator and discriminator.

—> **Why:**CNNs are very good at **handling images**, so DCGANs can create **more detailed and realistic pictures** than the basic GAN.

—> **Example:**If you train DCGAN on real photos of bedrooms, it can make **new, fake bedroom pictures** that look very realistic — with beds, lamps, and windows!

3️⃣ Conditional GAN (cGAN)

—> **What it is:**A **Conditional GAN** allows you to **control** what kind of image you want the generator to make.

—> **How it works:**You give the GAN some **extra information** (like a label or category).

—→ **Example:**If you train it on handwritten digits (0–9):

- You can say “Generate a 7” → it makes a fake 7.

So, you **decide what kind of fake image** it produces.

4 CycleGAN

—> **What it is:**A **CycleGAN** is used for **image-to-image translation** — it changes one type of image into another.

—> **Special feature:**It doesn’t need matching image pairs (unpaired translation).

—>**Example:**

- You have a set of **horse** photos and a set of **zebra** photos.
- It can learn to **turn a horse into a zebra** and **zebra into a horse** — without needing to match one horse with one zebra photo.

5 Pix2Pix GAN

—> **What it is:**Another image-to-image translation GAN — but **it needs paired data**.

—>Difference from CycleGAN: Pix2Pix **requires** that you have matching examples (like a before-and-after pair).

—> **Example:**

- A **black-and-white sketch** (input) and the **colored image** (target).
- The model learns to color any new sketch in a similar way.

6 StyleGAN

—>**What it is:**A **very advanced GAN** developed by NVIDIA.It can make **super-realistic human faces**, artworks, or other objects.

—> **Special feature:**It can control **different style levels** — like face shape, color, hair, background, and lighting — separately.

—> **Example:**The website **“This Person Does Not Exist”** uses StyleGAN to make **completely fake but realistic human faces**.

QUE:-2 Define the different layers of a Convolutional Neural Network (CNN).

ANS:- A **Convolutional Neural Network (CNN)** has several key layers that work together to extract features and classify images:

1. **Convolutional Layer:**Applies filters (kernels) to the input image to detect features like edges, colors, and textures. It produces **feature maps**.

2. **ReLU Layer (Activation Layer):**Introduces non-linearity by replacing negative values with zero. This helps the network learn complex patterns.

3. **Pooling Layer:**Reduces the size of feature maps (downsampling) to make computation faster and focus on the most important features.

4. **Fully Connected Layer:**Flattens the feature maps into a single vector and connects to neurons for **classification** (decides the final output).

THANK YOU

multiatoms